

Seminar 6

Matematica din spatele Transformatei Fourier Rapide

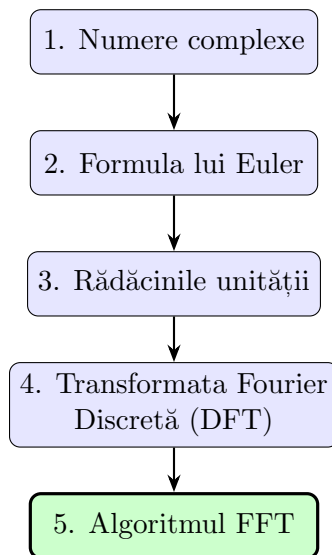
Asist. univ. dr. Mihai Tudor

23 martie 2026

1 Introducere și motivație

Transformata Fourier Rapidă (FFT — *Fast Fourier Transform*) este unul dintre cei mai importanți algoritmi din istoria calculatoarelor. Aplicațiile sale acoperă procesarea semnalelor audio și video, compresia de date (MP3, JPEG), analiza spectrală, rezolvarea ecuațiilor diferențiale și multe altele.

Înțelegerea FFT presupune stăpânirea mai multor concepte matematice care se construiesc unele pe altele:



2 Numere complexe — recapitulare și extindere

Definiție și reprezentare

Definiție 2.1. Un **număr complex** este o expresie de forma $z = a + bi$, unde $a, b \in \mathbb{R}$ și i este **unitatea imaginară**, definită prin $i^2 = -1$.

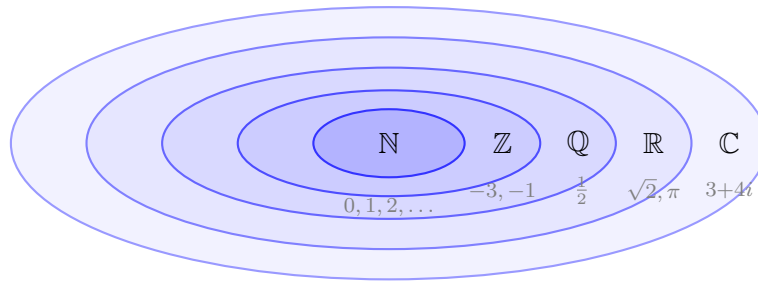
- $a = \operatorname{Re}(z)$ se numește **partea reală**
- $b = \operatorname{Im}(z)$ se numește **partea imaginară**
- **Modulul:** $|z| = \sqrt{a^2 + b^2}$
- **Conjugatul:** $\bar{z} = a - bi$

Definiție 2.2. Mulțimea numerelor complexe se notează $\mathbb{C}$:

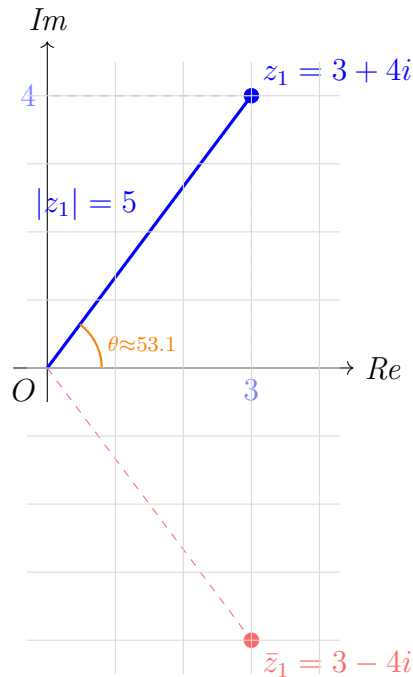
$$\mathbb{C} = \{a + bi \mid a, b \in \mathbb{R}, i^2 = -1\}$$

Relația de incluziune față de celelalte mulțimi numerice este:

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$$



Exemplu 2.1. Fie $z_1 = 3 + 4i$ și $z_2 = 1 - 2i$.



$$|z_1| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5 \quad \bar{z}_1 = 3 - 4i$$

$$z_1 \cdot z_2 = (3 + 4i)(1 - 2i) = 3 - 6i + 4i - 8i^2 = 3 - 2i + 8 = 11 - 2i$$

Operații esențiale

Operațiile cu numere complexe care vor fi folosite intensiv în FFT:

Operație	Formula	Exemplu cu $z_1 = 3 + 4i$, $z_2 = 1 - 2i$
Adunare	$(a + bi) + (c + di) = (a + c) + (b + d)i$	$(3 + 4i) + (1 - 2i) = 4 + 2i$
Scădere	$(a + bi) - (c + di) = (a - c) + (b - d)i$	$(3 + 4i) - (1 - 2i) = 2 + 6i$
Înmulțire	$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$	$(3 + 4i)(1 - 2i) = 11 - 2i$
Modul	$ a + bi = \sqrt{a^2 + b^2}$	$ 3 + 4i = 5$
Conjugat	$\overline{a + bi} = a - bi$	$\overline{3 + 4i} = 3 - 4i$
$ z ^2$	$z \cdot \bar{z} = a^2 + b^2$	$(3 + 4i)(3 - 4i) = 25$

3 Formula lui Euler

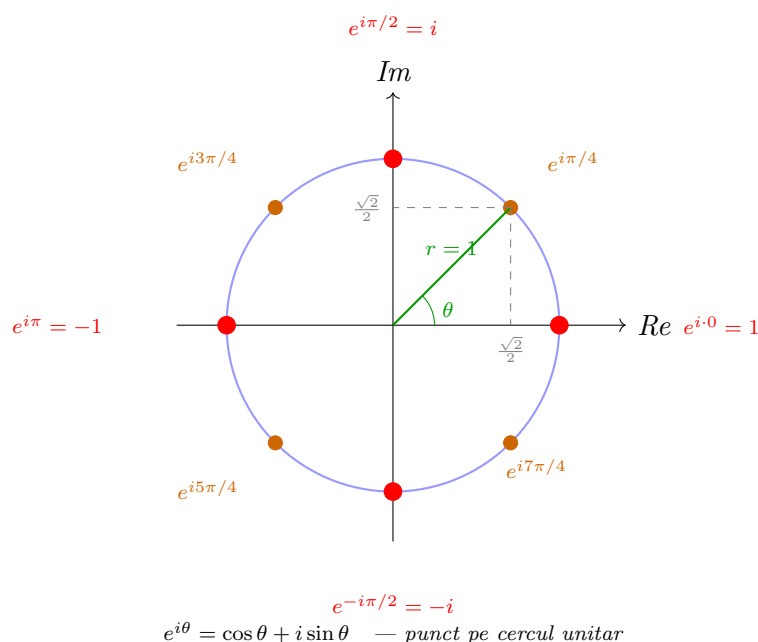
Enunț și semnificație geometrică

Teoremă 3.1 (Formula lui Euler). *Pentru orice unghi $\theta \in \mathbb{R}$:*

$$e^{i\theta} = \cos \theta + i \sin \theta$$

Aceasta leagă funcția exponențială complexă de funcțiile trigonometrice. Consecința imediată este că orice număr complex de modul 1 poate fi scris ca $e^{i\theta}$ pentru un unghi θ .

Exemplu 3.1. *Valorile canonice ale lui $e^{i\theta}$ — puncte pe cercul unitar:*



Tabelul valorilor exacte:

θ	$\cos \theta$	$\sin \theta$	$e^{i\theta}$
0	1	0	1
$\pi/6$	$\frac{\sqrt{3}}{2} \approx 0.866$	$\frac{1}{2}$	$\frac{\sqrt{3}}{2} + \frac{1}{2}i$
$\pi/4$	$\frac{\sqrt{2}}{2} \approx 0.707$	$\frac{\sqrt{2}}{2} \approx 0.707$	$\frac{\sqrt{2}}{2} + \frac{\sqrt{2}}{2}i$
$\pi/2$	0	1	i
π	-1	0	-1
$3\pi/2$	0	-1	$-i$
2π	1	0	1

Proprietăți cheie ale lui $e^{i\theta}$

Propoziție 3.1. *Fie $\theta, \phi \in \mathbb{R}$. Atunci:*

(1) $|e^{i\theta}| = 1$

$$(2) e^{i\theta} \cdot e^{i\phi} = e^{i(\theta+\phi)}$$

$$(3) \overline{e^{i\theta}} = e^{-i\theta}$$

$$(4) (e^{i\theta})^k = e^{ik\theta}$$

$$(5) e^{i\theta} = e^{i(\theta+2\pi k)} \text{ pentru orice } k \in \mathbb{Z} \quad (\text{periodicitate } 2\pi)$$

Observație 3.1. *De ce este formula lui Euler crucială pentru FFT?*

FFT se bazează pe rotații ale unor puncte pe cercul unitar. Expresia $e^{-2\pi i k/N}$ reprezintă o rotație de unghi $\frac{2\pi k}{N}$ în sens invers acelor de ceasornic. Toate calculele FFT se reduc la înmulțiri cu astfel de factori — numiți **factori twiddle**.

4 Rădăcinile N -a ale unității

Definiție 4.1. Rădăcina de ordin N a unității este:

$$W_N = e^{-2\pi i/N} = \cos\left(\frac{2\pi}{N}\right) - i \sin\left(\frac{2\pi}{N}\right)$$

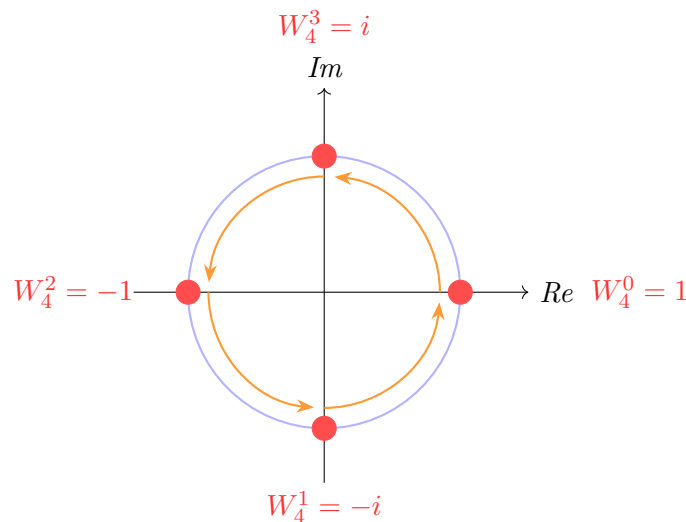
Cele N rădăcini de ordin N ale unității sunt:

$$W_N^k = e^{-2\pi i k/N}, \quad k = 0, 1, 2, \dots, N-1$$

Ele sunt soluțiile ecuației $z^N = 1$.

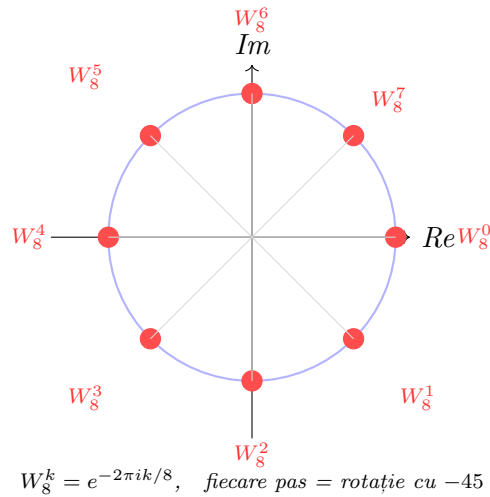
Exemplu 4.1. Rădăcinile de ordin 4 ale unității ($N = 4$):

$$W_4^0 = 1, \quad W_4^1 = -i, \quad W_4^2 = -1, \quad W_4^3 = i$$



Fiecare pas = rotație cu -90 ($= -\pi/2$)

Exemplu 4.2. Rădăcinile de ordin 8 ale unității ($N = 8$) — valori exacte:



k	$W_8^k = e^{-2\pi i k / 8}$	Parte reală	Parte imaginară
0	e^0	1	0
1	$e^{-i\pi/4}$	$\frac{\sqrt{2}}{2} \approx 0.707$	$-\frac{\sqrt{2}}{2} \approx -0.707$
2	$e^{-i\pi/2}$	0	-1
3	$e^{-i3\pi/4}$	$-\frac{\sqrt{2}}{2}$	$-\frac{\sqrt{2}}{2}$
4	$e^{-i\pi}$	-1	0
5	$e^{-i5\pi/4}$	$-\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$
6	$e^{-i3\pi/2}$	0	1
7	$e^{-i7\pi/4}$	$\frac{\sqrt{2}}{2}$	$\frac{\sqrt{2}}{2}$

Proprietăți fundamentale ale lui W_N^k

Aceste proprietăți sunt cele care fac posibilă reducerea complexității din $O(N^2)$ la $O(N \log N)$:

(1) **Periodicitate:** $W_N^{k+N} = W_N^k$

$$e^{-2\pi i(k+N)/N} = e^{-2\pi i k / N} \cdot e^{-2\pi i} = W_N^k \cdot 1 = W_N^k$$

(2) **Simetrie:** $W_N^{k+N/2} = -W_N^k$

$$e^{-2\pi i(k+N/2)/N} = e^{-2\pi i k / N} \cdot e^{-\pi i} = W_N^k \cdot (-1) = -W_N^k$$

(3) **Reducere:** $W_N^{2k} = W_{N/2}^k$

$$e^{-2\pi i(2k)/N} = e^{-2\pi i k / (N/2)} = W_{N/2}^k$$

(4) **Ortogonalitate:**

$$\sum_{n=0}^{N-1} W_N^{kn} = \begin{cases} N & \text{dacă } k \equiv 0 \pmod{N} \\ 0 & \text{altfel} \end{cases}$$

5 Implementări Python

Operații de bază cu numere complexe

```

1 import cmath
2 import numpy as np
3
4 # -----
5 # 1. Constructia si proprietatile unui numar complex
6 # -----
7 z1 = 3 + 4j          # Python foloseste j in loc de i
8 z2 = 1 - 2j
9
10 print(f"z1 = {z1}")
11 print(f"Re(z1) = {z1.real}")          # 3.0
12 print(f"Im(z1) = {z1.imag}")          # 4.0
13 print(f"|z1| = {abs(z1)}")            # 5.0
14 print(f"arg(z1)= {cmath.phase(z1):.4f} rad")    # 0.9273 rad
15 print(f"arg(z1)= {np.degrees(cmath.phase(z1)):.2f} grade") #
    53.13 grade
16 print(f"conj(z1) = {z1.conjugate()}")    # (3-4j)
17
18 # -----
19 # 2. Forma polara <-> forma carteziana
20 # -----
21 r, theta = cmath.polar(z1)              # z1 = r * e^(i*theta)
22 print(f"\nForma polara: r={r:.4f}, theta={theta:.4f} rad")
23
24 z_reconstruit = cmath.rect(r, theta) # r * e^(i*theta) -> a+bi
25 print(f"Reconstruit: {z_reconstruit:.4f}")    # (3+4j)
26
27 # -----
28 # 3. Formula lui Euler: e^(i*theta) = cos(theta)+i*sin(theta)
29 # -----
30 theta = np.pi / 4    # 45 grade
31 e_itheta = cmath.exp(1j * theta)
32 print(f"\ne^(i*pi/4) = {e_itheta:.4f}")
33 print(f"cos(pi/4) = {np.cos(theta):.4f}")
34 print(f"sin(pi/4) = {np.sin(theta):.4f}")
35
36 # Identitatea lui Euler: e^(i*pi) = -1
37 print(f"\ne^(i*pi) = {cmath.exp(1j * np.pi):.4f}")    # -1+0j
38
39 # -----
40 # 4. Operatii
41 # -----
42 print(f"\nz1 + z2 = {z1 + z2}")    # (4+2j)
43 print(f"z1 * z2 = {z1 * z2}")      # (11-2j)
44 print(f"|z1|*|z2| = {abs(z1)*abs(z2):.4f}")    # module se
    inmultesc

```

```

45 print(f"arg(z1)+arg(z2) = {np.degrees(cmath.phase(z1*z2)):.2f}
    grade")

```

Rădăcinile unității și vizualizare

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def radacinile_unitatii(N, afiseaza=True):
5      """
6      Calculeaza si (optional) afiseaza radacinile de ordin N ale
7      unitatii.
8      W_N^k = e^(-2*pi*i*k/N), k=0,...,N-1
9      """
10     k = np.arange(N)
11     W = np.exp(-2j * np.pi * k / N)
12
13     if afiseaza:
14         print(f"Radacinile de ordin {N} ale unitatii (W_{N}^k):")
15         for ki, w in enumerate(W):
16             print(f"  W_{N}^{ki} = {w.real:+.4f} {w.imag:+.4f}i")
17
18     return W
19
20 def plot_radacini(N):
21     """Vizualizeaza radacinile de ordin N pe cercul unitar."""
22     W = radacinile_unitatii(N, afiseaza=False)
23
24     fig, ax = plt.subplots(figsize=(6, 6))
25     # Cercul unitar
26     theta = np.linspace(0, 2*np.pi, 300)
27     ax.plot(np.cos(theta), np.sin(theta), 'b-', alpha=0.3,
28             linewidth=1)
29     # Axe
30     ax.axhline(0, color='gray', linewidth=0.5)
31     ax.axvline(0, color='gray', linewidth=0.5)
32     # Radacinile
33     ax.scatter(W.real, W.imag, color='red', zorder=5, s=60)
34     for k, w in enumerate(W):
35         ax.annotate(f'$W_{N}^{k}$',
36                   xy=(w.real, w.imag),
37                   xytext=(w.real*1.25, w.imag*1.25),
38                   ha='center', va='center', fontsize=9)
39     ax.plot([0, w.real], [0, w.imag], 'gray', linewidth=0.5,
40            alpha=0.5)
41     ax.set_xlim(-1.6, 1.6); ax.set_ylim(-1.6, 1.6)
42     ax.set_aspect('equal')
43     ax.set_title(f'Radacinile de ordin {N} ale unitatii')
44     ax.set_xlabel('Re'); ax.set_ylabel('Im')

```

```

43 plt.tight_layout()
44 plt.show()
45
46
47 # Demonstratii
48 W4 = radacinile_unitatii(4)
49 #  $W_4^0 = +1.0000 + 0.0000i$ 
50 #  $W_4^1 = +0.0000 - 1.0000i$  ( $= -i$ )
51 #  $W_4^2 = -1.0000 - 0.0000i$  ( $= -1$ )
52 #  $W_4^3 = -0.0000 + 1.0000i$  ( $= +i$ )
53
54 W8 = radacinile_unitatii(8)
55 plot_radacini(8) # Afiseaza grafic

```

Verificarea proprietăților pentru W_N^k

```

1 import numpy as np
2
3 def verifica_proprietati(N):
4     """
5     Verifica numeric toate proprietatile radacinilor unitatii  $W_N^k$ .
6     """
7     k_test = 2 # indice de test
8
9     print(f"== Proprietatile  $W_{\{N\}}^k$  ==\n")
10
11     # (1) Periodicitate:  $W_N^{(k+N)} = W_N^k$ 
12     w1 = np.exp(-2j * np.pi * k_test / N)
13     w2 = np.exp(-2j * np.pi * (k_test + N) / N)
14     print(f"(1) Periodicitate:")
15     print(f"     $W_{\{N\}}^{k\_test} = \{w1:.6f\}$ ")
16     print(f"     $W_{\{N\}}^{k\_test+N} = \{w2:.6f\}$ ")
17     print(f"    Egale? {np.isclose(w1, w2)}\n")
18
19     # (2) Simetrie:  $W_N^{(k + N/2)} = -W_N^k$ 
20     w3 = np.exp(-2j * np.pi * (k_test + N//2) / N)
21     print(f"(2) Simetrie (jumătate):")
22     print(f"     $W_{\{N\}}^{k\_test} = \{w1:.6f\}$ ")
23     print(f"     $W_{\{N\}}^{k\_test+N//2} = \{w3:.6f\}$ ")
24     print(f"     $-W_{\{N\}}^{k\_test} = \{-w1:.6f\}$ ")
25     print(f"     $W_{\{N\}}^{(k+N/2)} = -W_{\{N\}}^k?$  {np.isclose(w3, -w1)}\n")
26
27     # (3) Reducere:  $W_N^{(2k)} = W_{(N/2)}^k$ 
28     w4 = np.exp(-2j * np.pi * (2*k_test) / N)
29     w5 = np.exp(-2j * np.pi * k_test / (N//2))
30     print(f"(3) Reducere:")
31     print(f"     $W_{\{N\}}^{(2*\{k\_test\})} = \{w4:.6f\}$ ")
32     print(f"     $W_{\{N//2\}}^{k\_test} = \{w5:.6f\}$ ")

```

```

33     print(f"      Egale? {np.isclose(w4, w5)}\n")
34
35     # (4) Ortogonalitate:  $\sum W_N^{(k*n)} = 0$  pentru  $k \neq 0$ 
36     print(f"(4) Ortogonalitate:")
37     for k in [0, 1, 2, 3]:
38         suma = sum(np.exp(-2j*np.pi*k*n/N) for n in range(N))
39         print(f"      sum_n  $W_{\{N\}}^{(\{k\}*n)}$ , n=0..{N-1}: {suma:.4f}")
40
41
42 verifica_proprietati(8)

```

Vizualizare interactivă — forma trigonometrică

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  def vizualizeaza_forma_polara(numere_complexe, titlu="Numere
5      complexe"):
6      """
7      Vizualizeaza o lista de numere complexe in planul complex,
8      aratand modulul si argumentul fiecaruia.
9      """
10     fig, ax = plt.subplots(figsize=(7, 7))
11
12     # Cercul unitar ca referinta
13     theta = np.linspace(0, 2*np.pi, 300)
14     ax.plot(np.cos(theta), np.sin(theta), 'b--', alpha=0.2,
15             linewidth=1, label='Cerc unitar')
16
17     # Axe
18     ax.axhline(0, color='k', linewidth=0.8)
19     ax.axvline(0, color='k', linewidth=0.8)
20
21     culori = plt.cm.tab10(np.linspace(0, 1, len(numere_complexe))
22     )
23
24     for i, z in enumerate(numere_complexe):
25         r = abs(z)
26         theta_z = np.angle(z)
27         # Sageata de la origine
28         ax.annotate('', xy=(z.real, z.imag), xytext=(0, 0),
29                     arrowprops=dict(arrowstyle='->', color=culori
30                     [i],
31                     lw=2))
32
33         # Punct
34         ax.scatter(z.real, z.imag, color=culori[i], zorder=5, s
35                     =80)
36
37         # Eticheta
38         ax.annotate(
39             f'$z_{i+1}={z.real:.2f}{z.imag:+.2f}i$',

```

```

34         f'$r={r:.3f},\\theta={np.degrees(theta_z):.1f}$ ',
35         xy=(z.real, z.imag),
36         xytext=(z.real + 0.15, z.imag + 0.15),
37         fontsize=8, color=culori[i]
38     )
39
40     lim = max(abs(z) for z in numere_complexe) * 1.4 + 0.5
41     ax.set_xlim(-lim, lim); ax.set_ylim(-lim, lim)
42     ax.set_aspect('equal')
43     ax.set_xlabel('Re'); ax.set_ylabel('Im')
44     ax.set_title(titlu)
45     ax.grid(True, alpha=0.3)
46     plt.tight_layout()
47     plt.show()
48
49
50 # Exemplu de utilizare
51 numere = [3+4j, -1+1j, 2-2j, -2-1j]
52 vizualizeaza_forma_polara(numere, "Forma polara a numerelor
53     complexe")
54
55 # Animatie: rotatie progresiva
56 def animatie_rotatie(z0, n_pasi=8):
57     """Roteste z0 cu pasul 2*pi/n_pasi si arata fiecare pozitie."
58     """
59     rotatie = np.exp(2j * np.pi / n_pasi)
60     z_curent = z0
61     pozitii = []
62     for i in range(n_pasi):
63         pozitii.append(z_curent)
64         z_curent = z_curent * rotatie
65         vizualizeaza_forma_polara(pozitii, f"Rotatie in {n_pasi} pasi
66             ")
67
68 animatie_rotatie(1+0j, n_pasi=8)    # Genereaza  $W_8^k$  din 1

```